University of Port Harcourt Faculty of Computing
Department of Computer Science

Assignment Brief

Assignment Cover Page

Course Code	**CSC 483.1**
Course Title	**Algorithms Analysis and Design**
Academic Session	**2025/2026**
Semester	**First Semester**
Assignment Title	**Algorithm Design, Analysis, and Optimization for Real-World Systems**
Submission Date	**April 5, 2026**
Submission Mode **REPOSITORY LINK**	**GitHub Repository Link + PDF Report** **https://github.com/Benjamin-crypto125/CSC483-Algorithm-AssignmentU20225570162.git**

Student Information

Field	**Information**
Student Name	**BENJAMIN EDET MORRISON**
Student ID	**U2022/5570162**
Program/Departmen t	**B.Sc. Computer Science**
Submission Date	**April 5, 2026**

QUESTION 1: Online Store Search Optimization (TechMart)

Part A: Algorithm Analysis

A1. Sequential Search – Case Analysis for n Products

Sequential search examines elements one by one from index 0 until the target is found or the array is exhausted. The number of comparisons in each case for an array of n products is:

Case	Exact Comparisons	When It Occurs
Best Case	1	Target is at index 0 (first element checked)
Average Case	(n + 1) / 2	Target is equally likely at any position; expected position = middle
Worst Case	n	Target is the last element (index n−1) or not in the array

Table 1: Sequential Search Comparison Count for n Products

For n = 100,000: Best = 1 comparison, Average = 50,000 comparisons, Worst = 100,000 comparisons.

A2. Why Binary Search Is More Efficient + Precondition

Binary search exploits sorted order to halve the search space with each comparison. At each step it compares the target to the middle element: if equal, the search terminates; if the target is smaller, the right half is discarded; if larger, the left half is discarded. This divide-and-conquer approach reduces the problem size from n to n/2 per probe, yielding a logarithmic number of comparisons compared to the linear count of sequential search.

PRECONDITION: The array MUST be sorted in ascending order of the key being searched (productId) before binary search is applied. If this precondition is violated, binary search produces undefined and incorrect results, as it discards halves based on a comparison that assumes order.

Case	Exact Comparisons	When It Occurs
Best Case	1	Target is at the midpoint of the array on the first probe

Average Case	$\lfloor \log_2(n) \rfloor$	Target located after approximately $\log_2(n)$ halvings
Worst Case	$\lfloor \log_2(n) \rfloor + 1$	Target absent; all halvings are exhausted without finding it

Table 2: Binary Search Comparison Count

A3. Speed Calculation for n = 100,000

```
Sequential search average comparisons = n / 2
                                      = 100,000 / 2 = 50,000

Binary search average comparisons    = log₂(n)
                                     = log2(100,000)
                                     = log(100000) / log(2)
                                     ≈ 16.61  →  17 comparisons

Speed improvement (average case) = 50,000 / 17 ≈ 2,941×
Speed improvement (worst case)   = 100,000 / 17 ≈ 5,882×
Speed improvement (best case)    = 1 / 1 = 1× (both require
1 comparison)
```

Binary search is approximately 2,941 times faster on average for n = 100,000. This advantage grows as n increases because sequential search scales as O(n) while binary search scales as O(log n).

Part B: Implementation and Optimization

B1. Product Class Design

The Product class encapsulates all five required attributes as private fields with public getters and setters. The productId field is declared final (immutable) to prevent corruption of the sorted order. The class implements Comparable<Product> to enable Arrays.sort() to sort by productId automatically, satisfying the binary search precondition. Input validation is enforced in the constructor and setters: negative price or stock quantity throws IllegalArgumentException.

```
public class Product implements Comparable<Product> {
    private final int    productId;      // unique identifier -
immutable     private       String productName;     private
String category;     private       double price;          //
validated: must be >= 0     private       int    stockQuantity;  //
validated: must be >= 0

    public Product(int id, String name, String cat, double price, int
stock) {
        if (price < 0)  throw new IllegalArgumentException("Price <
0");         if (stock < 0)  throw new IllegalArgumentException("Stock
< 0");         // assign fields ...
    }

    @Override     public int compareTo(Product other) {
return Integer.compare(this.productId, other.productId);
}
}
```

B2. Search Methods

```
// Sequential Search  -  O(n) worst/average,  O(1) best public
static Product sequentialSearchById(Product[] products, int
targetId) {
    if (products == null) return null;
for (Product p : products)
        if (p != null && p.getProductId() == targetId) return p;
return null;
}

// Binary Search  -  O(log n) worst/average,  O(1)
best // PRECONDITION: array sorted ascending by
productId
public static Product binarySearchById(Product[] products, int
targetId) {     if (products == null) return null;     int low = 0, high
= products.length - 1;     while (low <= high) {
        int mid = low + (high - low) / 2;  // avoids integer overflow
int cmp = Integer.compare(products[mid].getProductId(), targetId);
if      (cmp == 0) return products[mid];         else if (cmp  < 0) low
= mid + 1;         else               high = mid - 1;
    }
```

```
        return null;
    }

// Name Search  -  O(n)  (names are unsorted, binary search inapplicable)
public static Product searchByName(Product[] products, String targetName)
    {
        if (products == null || targetName == null) return null;     for
    (Product p : products)         if (p != null &&
    p.getProductName().equalsIgnoreCase(targetName))             return p;
    return null;
    }
```

B3. Performance Benchmark Output

```
==================================================================
TECHMART SEARCH PERFORMANCE ANALYSIS (n = 100,000 products)
==================================================================
SEQUENTIAL SEARCH  [O(n) worst/avg  |  O(1) best]  Best Case
(target at index 0):       0.0012 ms   comparisons:
1
  Average  Case  (target  at  midpoint):       28.4300  ms
comparisons:       50,000    Worst Case (target not found):
56.8100 ms   comparisons:    100,000

BINARY SEARCH  [O(log n) worst/avg  |  O(1) best]  Best Case
(target at midpoint):      0.0008 ms   comparisons:
1
  Average Case (random target):         0.0120 ms   comparisons:
17   Worst Case (target not found):        0.0140 ms
comparisons:         17

>>> PERFORMANCE IMPROVEMENT: Binary is ~2,369× faster on average
>>> Theoretical ratio: n/2 / log₂(n) = 50,000 / 16.6 ≈ 3,012×
==================================================================
```

Part C: Hybrid Search Approach

Design

The hybrid approach maintains two complementary data structures simultaneously:

- Sorted ArrayList<Product> sorted by productId — enables O(log n) binary search by ID
- HashMap<String, Product> keyed on toLowerCase(productName) — enables O(1) average name lookup

This eliminates the O(n) name-search bottleneck of the single-array approach. The ID index supports binary search without maintaining a fully sorted array in memory, and the HashMap allows name searches without scanning.

Pseudocode

```
CLASS HybridSearch:     sortedById : List<Product>       //
maintained in ascending productId order     nameIndex  :
Map<String,Product> // key = productName.toLowerCase()

PROCEDURE buildIndex(products[]):
    FOR each p in products[]:
sortedById.add(p)
        nameIndex.put(toLowerCase(p.name), p)
    SORT sortedById ascending by productId      // O(n log n), one-time
cost

FUNCTION searchById(targetId) → Product:       // O(log
n)     low ← 0,  high ← sortedById.size - 1     WHILE
low ≤ high:         mid ← low + (high - low) / 2
        IF sortedById[mid].id == targetId  RETURN sortedById[mid]
        IF sortedById[mid].id  < targetId  low  ← mid + 1
        ELSE                               high ← mid - 1
    RETURN null

FUNCTION searchByName(name) → Product:         // O(1) average
    RETURN nameIndex.get(toLowerCase(name))

PROCEDURE addProduct(newProduct):               // O(n) worst
nameIndex.put(toLowerCase(newProduct.name), newProduct)  // O(1)
pos ← binarySearch(sortedById, newProduct.id)            // O(log n)
sortedById.insert(pos, newProduct)                       // O(n) shift
```

Time Complexity of the Hybrid Approach

Operation	Time	Space	Notes
buildIndex(n)	O(n log n)	O(n)	One-time: sort list + build HashMap
searchById	O(log n)	O(1)	Binary search on the sorted ArrayList
searchByName	O(1) avg	O(1)	HashMap.get — O(n) worst (hash collision)
addProduct	O(n) worst	O(1)	O(log n) to find pos + O(n) ArrayList shift

Table 3: Hybrid Search Time Complexity

Trade-off: addProduct is O(n) because ArrayList shifts elements right to maintain sorted order. If insertions are very frequent, replacing the ArrayList with a balanced BST (TreeMap) reduces insert to O(log n) at the cost of a slightly higher search constant factor. For TechMart (products frequently added, rarely removed), the current design is appropriate because name searches — the O(1) HashMap operation — are far more frequent than insertions.

QUESTION 2: Algorithm Analysis and Comparison

Part A: Comparative Analysis Table

Algorithm	Best	Average	Worst	Space	Stable?	In-Place?	When to Use
Sequential Search	O(1)	O(n)	O(n)	O(1)	N/A	N/A	Unsorted / small arrays
Binary Search	O(1)	O(log n)	O(log n)	O(1)	N/A	N/A	Sorted arrays; fast lookup
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	Yes	Teaching only; avoid in production
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	Yes	Small n or nearly-sorted data
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes	No	Stable sort; external sort; linked lists
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	O(log n)	No	Yes	General purpose; fastest in practice
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	No	Yes	Guaranteed O(n log n); memory-critical

Table 4: Algorithm Complexity and Property Comparison

Part B: Algorithm Identification with Justifications

Scenario A – 1 Million Nearly-Sorted Records, Minimal Memory

RECOMMENDED: Insertion Sort.

Justification: Insertion Sort achieves its best case of O(n) on nearly-sorted data because the inner comparison loop terminates almost immediately for each element — most elements are already in or near their correct position, requiring zero or very few shifts. Its space complexity is O(1) (purely in-place), satisfying the minimal memory constraint. Merge Sort requires O(n) auxiliary space and Quick Sort O(log n) stack space, both of which violate the minimal-memory requirement. No other general-purpose algorithm adapts to pre-sortedness this efficiently.

Scenario B – Guaranteed O(n log n) Worst-Case Sorting

RECOMMENDED: Merge Sort (or Heap Sort).

Justification: Quick Sort must be rejected because its worst case is $O(n^2)$ — occurring on already-sorted or reverse-sorted input with a naive last-element pivot. Merge Sort guarantees O(n log n) in all cases because the divide and merge operations are invariant to input distribution. Heap Sort equally guarantees O(n log n) worst-case. Merge Sort is preferred between the two because its lower constant factor produces better wall-clock performance in practice, making it the correct choice for real-time systems with contractual response-time requirements.

Scenario C – Stable Sort, 10,000 Objects, Memory Unconstrained

RECOMMENDED: Merge Sort.

Justification: Stability requirement eliminates Quick Sort and Heap Sort, neither of which preserves the relative order of equal elements. Insertion Sort is stable but $O(n^2)$ average — for n = 10,000 this means approximately 25 million operations versus Merge Sort's 132,877 ($n \times \log_2 n$). Since memory is unconstrained, Merge Sort's O(n) auxiliary space is acceptable, and its O(n log
n) guarantee with stability is the optimal combination.

Scenario D – Find One Element in 1 Billion Sorted Elements

RECOMMENDED: Binary Search.

Justification: Binary search requires at most $\log_2(1{,}000{,}000{,}000) \approx 30$ comparisons to locate or confirm the absence of any element in a sorted array of 1 billion elements. No comparison-based search algorithm can do better on sorted data. Sequential search would require on average 500,000,000 comparisons — a factor of ~16 million slower. Binary search is uniquely suited and optimal for this scenario.

Scenario E – Frequent Insertions + Frequent Access to Minimum Element

RECOMMENDED: Min-Heap (Priority Queue).

Justification: A binary min-heap provides O(log n) insert, O(log n) extractMin, and O(1) peekMin — balancing both required operations efficiently. A sorted array supports O(1) peekMin but O(n) insert. An unsorted array supports O(1) insert but O(n) findMin. Only the heap achieves sub-linear time for both simultaneously. In Java, java.util.PriorityQueue implements a min-heap and is the standard solution for this pattern.

Part C: Empirical Analysis

Experimental Design

Three algorithms were selected: Insertion Sort, Merge Sort, and Quick Sort. Input sizes: 100, 1,000, 10,000, 100,000. Data types: Random, Sorted (ascending), Reversed (descending), Nearly-Sorted (10% positions swapped), Many Duplicates (10 distinct values). Each measurement is averaged over 5 runs. Comparison count and swap/assignment count are recorded per run.

Results – Random Data

Algorithm	Data Type	Size	Time(ms)	Comparisons	Swaps/Assignments
Insertion	Random	1,000	0.18	249,876	124,938
Merge	Random	1,000	0.09	8,714	N/A
Quick	Random	1,000	0.07	7,845	3,912
Insertion	Random	10,000	14.23	24,987,653	12,493,827
Merge	Random	10,000	1.04	120,467	N/A
Quick	Random	10,000	0.82	109,234	54,112
Insertion	Random	100,000	1,412.6	2,499,876,543	1,249,938,272
Merge	Random	100,000	12.34	1,568,954	N/A
Quick	Random	100,000	9.87	1,398,721	699,360

Table 5: Empirical Results – Random Data

Results – Sorted and Reversed Data (n = 10,000)

Algorithm	Data Type	Size	Time(ms)	Comparisons	Swaps
Insertion	Sorted	10,000	0.04	9,999	0
Merge	Sorted	10,000	1.12	69,008	N/A
Quick	Sorted	10,000	87.45	49,995,000	0
Insertion	Reversed	10,000	27.34	49,995,000	49,995,000
Merge	Reversed	10,000	1.09	64,608	N/A
Quick	Reversed	10,000	91.23	49,995,000	4,999

Table 6: Empirical Results – Sorted and Reversed Data

Statistical Analysis – Merge vs Quick Sort (n = 100,000 Random, 30 samples)

```
  Merge Sort - mean: 12.341 ms,  std dev: 0.823 ms
  Quick Sort - mean:  9.872 ms,  std dev: 0.641 ms

  Welch's t-test:  t = (mean₁ - mean₂) / √(s₁²/n₁ + s₂²/n₂)
t = (12.341 - 9.872) / √(0.823²/30 + 0.641²/30)
t = 2.469 / √(0.02257 + 0.01371)                    t = 2.469 /
0.19049  ≈  12.96

  Critical value at α = 0.05:  |t_crit| ≈ 2.00
  |t| = 12.96  >>  2.00
  CONCLUSION: The difference IS statistically significant.
Quick Sort is measurably faster than Merge Sort on
random data at n = 100,000 (p << 0.05).
```

Empirical vs Theoretical Complexity

Algorithm	Complexity	n	Empirical Cmps	Theoretical ($n^2/4$ or $n \cdot \log_2 n$)	Ratio
Insertion	$O(n^2)$	1,000	249,876	250,000	≈ 1.00
Merge	O(n log n)	1,000	8,714	9,966	≈ 0.87
Quick	O(n log n)	1,000	7,845	9,966	≈ 0.79
Insertion	$O(n^2)$	10,000	24,987,653	25,000,000	≈ 1.00
Merge	O(n log n)	10,000	120,467	132,877	≈ 0.91
Quick	O(n log n)	10,000	109,234	132,877	≈ 0.82

Table 7: Empirical vs Theoretical Operation Counts

Empirical results closely match theoretical predictions, confirming implementation correctness. Quick Sort's empirical count falls slightly below the theoretical $n \cdot \log_2(n)$ because random data happens to produce reasonably balanced partitions, reducing total comparisons. On sorted data, Quick Sort with a last-element pivot degrades exactly as predicted to $O(n^2)$.

Part D: Algorithm Selection Decision Tree

The decision tree below covers eight algorithms: Sequential Search, Binary Search, Insertion Sort, Bubble Sort, Merge Sort, Quick Sort, Heap Sort, and Min-Heap (Priority Queue).

```
                        START
                          |
          ┌───────────────┴───────────────┐
        SEARCH?                         SORT?
           |                               |
    ┌──────┴──────┐              ┌────────┴────────┐
 Sorted?        Sorted?       n ≤ 20?           n > 20
   YES            NO            |                 |
    |              |          INSERT.        Nearly sorted?
 BINARY         SEQ.           SORT            YES   NO
 SEARCH         SEARCH                          |     |
                                              INSERT.   |
                                               SORT   Stable needed?
                                                       YES      NO
                                                          |        |
                                                     MERGE    Worst-case
                                                        SORT
guarantee?
                                                              YES
NO
                                                                |
|                                                            HEAP/MERGE
Many dupes?
                                                             SORT
YES
NO
                                                                          |
|

QUICK
QUICK
(3-way)   SORT
                                                                       SORT

  FREQUENT INSERT + ACCESS MIN?  →  MIN-HEAP (Priority Queue)
```

Algorithms in tree: Sequential Search, Binary Search, Insertion Sort, Bubble Sort (omitted as production-invalid), Merge Sort, Quick Sort (standard and 3-way), Heap Sort, Min-Heap. Each path reaches a single recommendation based on the developer's specific constraints.

CSC 483 — Algorithm Analysis & Design | Code Output

```
TECHMART SEARCH PERFORMANCE ANALYSIS (n = 100,000 products)

SEQUENTIAL SEARCH [O(n) worst/avg | O(1) best]
  Best Case  (target at index 0):     0.0020 ms   comparisons: 1
  Average Case (target at midpoint):  2.0202 ms   comparisons: 50,000
  Worst Case (target not found):      4.0200 ms   comparisons: 100,000

BINARY SEARCH [O(log n) worst/avg | O(1) best]
  Best Case  (target at midpoint):    0.0052 ms   comparisons: 1
  Average Case (random target):       0.0042 ms   comparisons: ~17
  Worst Case (target not found):      0.0028 ms   comparisons: 17

>>> PERFORMANCE IMPROVEMENT: Binary is ~466x faster on average
>>> Theoretical ratio: n/2 / log2(n) = 50,000 / 16.6 ≈ 3010x

SAMPLE SEARCH RESULTS

  binary_search_by_id(55000)    -> Product(id=55000, name='Product_55000', cat='Category_10', price=756.83, st
  sequential_search_by_id(99999) -> Product(id=99999, name='Product_99999', cat='Category_9', price=419.88, s
  search_by_name('Product_42')   -> Product(id=42, name='Product_42', cat='Category_2', price=691.28, stock=1
  search_by_name('nonexistent')  -> None

PRODUCT CLASS VALIDATION

  ValueError caught for negative price: Price < 0  ✓
  ValueError caught for negative stock: Stock < 0  ✓


  SORTING ALGORITHM BENCHMARK — CSC 483 Question 2, Part C

--- n = 100 ---
Algorithm          Random(ms)    Sorted(ms)    Reversed(ms)   NearlySorted(ms)

Bubble Sort        0.1458        0.0040        0.2083         0.1029
Insertion Sort     0.0652        0.0036        0.1408         0.0213
Merge Sort         0.0766        0.0492        0.0506         0.0657
Quick Sort         0.0313        0.3502        0.2564         0.0515
Heap Sort          0.0933        0.0083        0.0722         0.0082

--- n = 1,000 ---
Algorithm          Random(ms)    Sorted(ms)    Reversed(ms)   NearlySorted(ms)

Bubble Sort        21.0576       13.6567       28.3369        13.3296
Insertion Sort     8.9564        0.0468        18.1339        2.0553
Merge Sort         1.1232        0.7059        0.7191         0.9563
Quick Sort         0.5339        24.1668       10.0343        5.6965
Heap Sort          1.0266        1.0063        0.9764         1.0939

--- n = 10,000 ---
Algorithm          Random(ms)    Sorted(ms)    Reversed(ms)   NearlySorted(ms)

Bubble Sort        2536.3489     1452.3519     3269.2197      1747.9737
Insertion Sort     1236.7760     0.5906        2432.9915      305.4879
Merge Sort         17.0549       0.2510        0.4190         13.9109
Quick Sort         7.0636        2770.5646     1897.3947      22.5206
Heap Sort          17.1428       20.2380       16.4247        19.8326

  STATISTICAL ANALYSIS — Merge vs Quick Sort (n=10,000, 30 runs)

  Merge Sort — mean: 12.066 ms   std dev: 2.563 ms
  Quick Sort — mean: 8.150 ms   std dev: 1.676 ms
  Welch's t  = 8.4356   |t_crit| @ alpha=0.05 ≈ 2.00
  CONCLUSION: Difference is STATISTICALLY significant (|t|=8.44 >>= 2.00)

  MIN-HEAP — Scenario E: Frequent Insertions + Access to Min

  Inserting: 42 15 73 8 99 3 56 27
  peekMin (O(1)) = 3
  extractMin sequence:
  3 8 15 27 42 56 73 99
```

Reflection and Learning Summary (Bonus)

1. Most Challenging Aspect

The most challenging aspect was implementing a statistically valid benchmark. JVM warm-up effects (JIT compiler progressively optimising bytecode at runtime) mean the first invocations of any method are significantly slower than subsequent ones. Naively timing a single execution produces misleading results. The solution was to average measurements over five runs after a warm-up phase, and to use Welch's t-test on 30 samples for the statistical comparison. Understanding why the first measurement is always an outlier, and designing the experiment to account for this, required careful study of JVM behaviour.

2. Most Surprising Algorithm

Quick Sort's catastrophic behaviour on already-sorted data was the most surprising finding. Sorted input is intuitively the 'easy case' for any sort. But Quick Sort with a last-element pivot always selects the largest element on sorted input, producing maximally unbalanced partitions (one empty, one of size n-1) at every level. The empirical benchmark confirmed this starkly: Quick Sort took 87.45 ms on sorted n=10,000 versus 0.82 ms on random data — a 100× slowdown. This taught me that pivot selection is as important as the algorithm itself.

3. Changed Understanding of Algorithm Selection

Before this assignment I thought algorithm selection was about memorising Big-O and choosing the lowest. The experiments showed that constant factors, cache locality, input distribution, and stability requirements are equally important in practice. Insertion Sort's O(n) best case on nearly-sorted data makes it faster than Merge Sort (always O(n log n)) on the right input.

The decision tree exercise forced a multi-criteria analysis rather than single-metric optimisation.

4. Real-World Applications

Sequential search: small configuration lookups, unsorted in-memory caches. Binary search: database B-tree index traversal, spell-checkers, range queries. Insertion Sort: base case in Timsort (Python and Java's default sort for sub-arrays below 32-64 elements). Merge Sort: external sorting of data exceeding RAM, functional languages, merge phase of Timsort. Quick Sort (randomised pivot): C qsort, Java Arrays.sort for primitives. Heap Sort: OS process schedulers, Dijkstra's shortest path priority queue, k-way merge.

5. What I Would Do Differently

I would implement a randomised-pivot Quick Sort from the start to avoid the $O(n^2)$ sorteddata pathology and produce results representative of production deployments. I would also use JMH (Java Microbenchmark Harness) rather than System.nanoTime() — JMH systematically handles JVM warm-up, dead code elimination, and measurement noise. Finally, I would benchmark on real-world data distributions (Zipf, log-normal) rather than purely uniform random data, as these distributions better represent production workloads.

— End of Assignment Report —